ΑΝΑΛΥΤΙΚΕΣ ΟΔΗΓΙΕΣ · ΕΚΔΟΣΗ 2.0

SOL-TRADE Τακτοποίηση

Πλήρης έλεγχος του repository markos-kapsalos/SOL-TRADE: ποια αρχεία διαγράφονται, ποια διορθώνονται και ποια αντικαθίστανται — μαζί με βήμα-βήμα οδηγό για paper trading με πραγματικές τιμές Jupiter, τοπικά ή στο GitHub Actions.

makiss1956

markos-kapsalos-SOL-TRADE · Trading bot για Solana

ΝΟΕΜΒΡΙΟΣ 2026

Περιεχόμενα

1. Τι βρέθηκε στον έλεγχο 1

2. Ποια αρχεία σβήνουμε 2

3. Ποια αρχεία διορθώνουμε 3

4. Τι αντικαθίσταται και τι προστίθεται 4

5. Η νέα δομή και η μετάβαση στο GitHub 5

6. Paper trading τοπικά — βήμα προς βήμα 6

7. Paper trading στο GitHub Actions (χωρίς ανοιχτό PC) 7

8. Πώς διαβάζουμε τα αποτελέσματα 8

9. Η πορεία προς live mode (προσεκτικά) 8

10. Συχνές ερωτήσεις και προβλήματα 9

1. Τι βρέθηκε στον έλεγχο

Έκανα πλήρη κλωνοποίηση και επιθεώρηση του repository **makiss1956/markos-kapsalos-SOL-TRADE**: κάθε αρχείο διαβάστηκε, συντάχθηκε και εκτελέστηκε. Η κεντρική διαπίστωση είναι ότι στο repo συνυπάρχουν **δύο διαφορετικά bots** που μπερδεύονται μεταξύ τους. Το πρώτο είναι το σωστό, πλήρες σύστημα (Jupiter API, EMA/RSI στρατηγική, stop-loss/take-profit, backtesting, 23 unit tests) που όμως **δεν μπορεί να τρέξει καθόλου** στη σπαρμένη μορφή του, γιατί τα αρχεία του πακέτου **sol_trade/** μεταφέρθηκαν λάθος στη ρίζα του repo. Το δεύτερο είναι ένα απλοϊκό prototype μέσα στον φάκελο **src/** (με CoinGecko αντί για Jupiter), το οποίο εκτελείται από ένα GitHub Actions workflow κάθε δύο ώρες — αλλά με άχρηστο αποτέλεσμα, όπως φαίνεται παρακάτω.

2
διαφορετικά bots στο ίδιο repo

0/23
tests που περνούσαν (σπασμένα imports)

165/0
αγορές χωρίς ΟΥΤΕ ΜΙΑ πώληση στο CSV

Το αρχείο **data/trade_history.csv** το αποδεικνύει πλήρως: από τις 13.10.2026 έως τις 04.11.2026 καταγράφηκαν 165 αγορές (BUY) περίπου 5 USD η καθεμία και **καμία πώληση**. Ο λόγος είναι διπλός: η παλιά στρατηγική στο src/strategy/basic_strategy.py επιστρέφει μόνο BUY ή HOLD (ποτέ SELL), και κάθε εκτέλεση του workflow ξεκινάει με φρέσκο υπόλοιπο 1.000 USD, οπότε το χαρτοφυλάκιο μηδενίζεται συνεχώς και κανένα κέρδος δεν συσσωρεύεται. Δηλαδή το «paper trading» που έτρεχε στο GitHub επί τρεις εβδομάδες **δεν μετρούσε πραγματικά τίποτα**. Αυτό διορθώνεται οριστικά με τη νέα δομή.

1.1 Τα πέντε βασικά ευρήματα

#	Εύρημα	Σοβαρότητα	Επίπτωση
1	Τα 16 αρχεία του πακέτου sol_trade/ βρίσκονται στη ρίζα· το run.py ζητάει «from sol_trade...» που δεν υπάρχει	ΚΡΙΣΙΜΟ	ModuleNotFoundError — ούτε το bot ούτε τα tests ξεκινούν
2	Δεύτερο παλιό bot μέσα στο src/ με δικό του risk/strategy/paper (CoinGecko, στατικά όρια)	ΥΨΗΛΟ	Διπλή, συγκρουόμενη υλοποίηση· μπέρδεμα και συντήρηση δύο κωδικών
3	GitHub Action «trade_bot.yml» εκτελούσε το παλιό bot κάθε 2 ώρες και κάνει commit το CSV	ΚΡΙΣΙΜΟ	Άχρηστο paper trading: μηδενισμοί, μόνο αγορές, κόστος σε Actions minutes
4	Λείπει το .env.example που παραπέμπει το README	ΜΕΣΟ	Ο νέος χρήστης δεν ξέρει ποια κλειδιά χρειάζονται

#	Εύρημα	Σοβαρότητα	Επίπτωση
5	Διάσπαρτα σφάλματα: διπλό import στο backtest.py, bug «opened_at» στο live_engine.py, κατεστραμμένα ονόματα (,gitkeep), ιαπωνικοί χαρακτήρες στο workflow	ΜΕΣΟ	Οπτικός θόρυβος και λανθασμένα δεδομένα στο live mode

Πίνακας 1: Ευρήματα ελέγχου με σοβαρότητα και επίπτωση

Σημαντικό: **δεν βρέθηκε κανένα μυστικό ή κλειδί** μέσα στο repo — το .gitignore καλύπτει σωστά το .env και το CSV δεν περιέχει ευαίσθητα δεδομένα. Ο έλεγχος ασφάλειας πέρασε καθαρά, οπότε δεν χρειάζεται ανάκληση κλειδιών.

2. Ποια αρχεία σβήνουμε

Παρακάτω είναι η πλήρης λίστα διαγραφής. Όλα αυτά τα αρχεία είναι είτε διπλότυπα (υπάρχει καλύτερη έκδοσή τους στο νέο πακέτο), είτε κατεστραμμένα, είτε επικίνδυνα. Δεν χάνετε τίποτα: το παλιό ιστορικό συναλλαγών αρχειοθετείται στον φάκελο **archive/** αντί να διαγραφεί οριστικά.

Αρχείο / φάκελος	Τι είναι	Γιατί σβήνεται
src/ (ολόκληρος)	Το παλιό prototype bot (main.py, fetcher.py, manager.py, basic_strategy.py)	Αντικαθίσταται πλήρως από το σωστό πακέτο sol_trade/ — δεν μιλάει καν με Jupiter
config/settings.py	Παλιές στατικές ρυθμίσεις (devnet, όρια)	Αντικαθίσταται από config.yaml + sol_trade/config.py
paper_trading/simulator.py	Παλιός προσομοιωτής με CSV	Αντικαθίσταται από sol_trade/paper_engine.py (state σε JSON, επιβιώνει από επανεκκινήσεις)
scripts/scripts/view_history.py	Διπλή φωλιά φακέλων· άλλωστε σκάει στο πραγματικό CSV (ασυμφωνία στηλών)	Αντικαθίσταται από την εντολή python run.py status
.github/workflows/trade_bot.yml	Workflow που έτρεχε το παλιό bot κάθε 2 ώρες και κάνει commits	Εκτελούσε λάθος bot με μηδενισμούς· αντικαθίσταται από paper-trading.yml
docs/README.md, docs/ROADMAP.md	Κενά stub αρχεία	Αντικαθίστανται από τον αναλυτικό οδηγό PDF στα docs/
.github/,gitkeep · paper_trading/,gutkeep · tests/,gitkeep · src/notifications/,gitkeep	Κατεστραμμένα placeholder αρχεία (κόμμα αντί τελείας)	Αντικείμενα σκουπιδιών από λάθος upload
__init__.py (στη ρίζα)	Ο δείκτης του πακέτου σε λάθος θέση	Μετακινείται μέσα στο sol_trade/ όπου ανήκει

Αρχείο / φάκελος	Τι είναι	Γιατί σβήνεται
data/trade_history.csv	Ιστορικό του παλιού bot: 165 BUY, χωρίς επικεφαλίδα, ασυνεπείς στήλες	Αρχειοθετείται ως archive/old_bot_history_2026-10.csv

Πίνακας 2: Πλήρης λίστα διαγραφών με αιτιολόγηση

> **Προσοχή πριν τη μετάβαση:** όσο υπάρχει το παλιό trade_bot.yml στο repo, το GitHub θα συνεχίσει να τρέχει το λάθος bot. Αν κάνετε τη μετάβαση με τον τρόπο «πλήρης αντικατάσταση» (Κεφ. 5), το πρόβλημα λύνεται από μόνο του γιατί το αρχείο διαγράφεται μαζί με όλα τα υπόλοιπα.

3. Ποια αρχεία διορθώνουμε

Ο βασικός κώδικας του κυρίως bot είναι υψηλού επιπέδου και κρατήθηκε σχεδόν ολόκληρος. Οι διορθώσεις που έγιναν είναι στοχευμένες και όλες έχουν επιβεβαιωθεί με εκτέλεση: 23/23 unit tests περνούν, το integration test ολοκληρώνει συναλλαγές, και ένα πραγματικό tick με live τιμή Jupiter επέστρεψε SOL στα 92,68 USD.

Αρχείο	Πρόβλημα που βρέθηκε	Η διόρθωση που εφαρμόστηκε
run.py	Έλειπε το πακέτο sol_trade/ που ζητούν τα imports (και ένα αχρησιμο import utc_now_iso)	Επαναφέρουμε τον φάκελο sol_trade/ με τα 16 modules μέσα του — το run.py παύει να χρειάζεται αλλαγή
sol_trade/backtest.py	Διπλή εισαγωγή από indicators και αχρηστο σύμβολο last_of	Καθαρίστηκε το μπλοκ imports
sol_trade/live_engine.py	Στο αποτέλεσμα αγοράς η τιμή opened_at γέμιζε με το κείμενο της αιτίας (bug)	Τώρα γράφει σωστή UTC ώρα με utc_now_iso()
sol_trade/jupiter_client.py	Σκληρά δεμένα URLs στο api.jup.ag που χωρίς κλειδί κινδυνεύει από rate limits	Αυτόματη επιλογή: χωρίς κλειδί χρησιμοποιεί το δωρεάν lite-api.jup.ag, με κλειδί το api.jup.ag· προστέθηκε και παράμετρος base_url
sol_trade/bot.py	Το ιστορικό τιμών ζούσε μόνο στη μνήμη — σε κάθε επανεκκίνηση ή Action run ξεκινούσε από το μηδέν	Αποθήκευση σε data/price_history.json μετά από κάθε tick και φόρτωση στην εκκίνηση· επιπλέον ο βρόχος πλέον δεν «πεθαίνει» σε προσωρινό σφάλμα δικτύου
sol_trade/paper_engine.py	Το paper_state.json δημιουργούνταν μόνο στο πρώτο trade — το status έλεγε «δεν υπάρχει κατάσταση»	Άμεση δημιουργία state στην πρώτη εκκίνηση
sol_trade/config.py	Ασαφές KeyError όταν το config.yaml λείπει ή είναι κατεστραμμένο	Φιλικό ελληνικό μήνυμα που εξηγεί τη λύση

Πίνακας 3: Διορθώσεις ανά αρχείο

Η σημαντικότερη βελτίωση είναι η **μονιμοποίηση ιστορικού τιμών**. Χωρίς αυτήν, ένα bot που τρέχει ένα βήμα ανά ώρα στο GitHub Actions δεν θα μαζεύει ποτέ αρκετές τιμές για να «ζεσταθούν» οι δείκτες EMA/RSI. Με την αλλαγή, κάθε νέο tick προσθέτεται στο ίδιο συνεχές ιστορικό, οπότε το paper trading στο Actions γίνεται πλέον ουσιαστικά ισοδύναμο με το τοπικό.

4. Τι αντικαθίσταται και τι προστίθεται

Αρχείο	Ενέργεια	Σημείωση
.env.example	ΠΡΟΣΘΗΚΕ	Νέο: πρότυπο με όλα τα κλειδιά και οδηγίες στα ελληνικά — αντιγράφετέ το σε .env
.github/workflows/paper-trading.yml	ΠΡΟΣΘΕΚΕ (αντικαθιστά το trade_bot.yml)	Τρέχει ΕΝΑ βήμα paper trading κάθε ώρα με πραγματικές τιμές Jupiter και αποθηκεύει το state πίσω στο repo
README.md	ΕΝΗΜΕΡΩΘΗΚΕ	Νέα δομή φακέλων, ενότητα για GitHub Actions, διορθωμένες αναφορές
docs/Sol-Trade-Odhgos.pdf	ΠΡΟΣΘΕΚΕ	Ο παρών αναλυτικός οδηγός μέσα στο repo
archive/old_bot_history_2026-10.csv	ΠΡΟΣΘΕΚΕ	Το παλιό ιστορικό των 165 αγορών, για αναφορά
SECURITY.md	ΠΑΡΑΜΕΝΕΙ	Σωστό ως έχει
.gitignore, config.yaml, requirements.txt, tests/, integration_test.py	ΠΑΡΑΜΕΝΟΥΝ	Άθικτα — πέρασαν πλήρως όλους τους ελέγχους

Πίνακας 4: Νέα και ενημερωμένα αρχεία

Το νέο workflow **paper-trading.yml** είναι ριζικά διαφορετικό από το παλιό: χρησιμοποιεί το σωστό bot (run.py trade --once), διατηρεί το υπόλοιπο και το ιστορικό μεταξύ εκτελέσεων (κανένας μηδενισμός), σταματά να κάνει commits όταν τίποτα δεν άλλαξε, και υποστηρίζει Telegram ειδοποιήσεις μέσω GitHub Secrets. Μπορείτε να το αφήσετε ανενεργό — τρέχει μόνο αν το ενεργοποιήσετε εσείς από το tab Actions.

5. Η νέα δομή και η μετάβαση στο GitHub

Νέα δομή φακέλων

```
sol_trade/
├── run.py # Είσοδος: trade / backtest / status / reset
├── integration_test.py # End-to-end δοκιμή
├── config.yaml # Στρατηγική + ρίσκο (το επεξεργάζεστε εσείς)
├── .env.example # Πρότυπο κλειδιών -> αντιγραφή σε .env
├── requirements.txt
├── README.md · SECURITY.md · .gitignore
├── sol_trade/ # ΤΟ πακέτο (επαναφέρθηκε!)
│ ├── bot.py config.py jupiter_client.py data_feed.py
│ ├── indicators.py strategy.py risk.py
│ ├── paper_engine.py live_engine.py wallet.py
│ └── notifier.py backtest.py logger.py utils.py
├── tests/ # 23 unit tests (pytest)
├── .github/workflows/paper-trading.yml # Προαιρετικό hourly paper tick
├── archive/ # Παλιά δεδομένα (αναφορά)
├── docs/ # Ο παρών οδηγός (PDF)
└── data/ # Δημιουργείται: state, ιστορικό, logs
```

5.1 Τρόπος Α — Πλήρης αντικατάσταση από το web (πιο απλός)

- Κατεβάστε το νέο πακέτο (sol_trade.zip) και αποσυμπιέστε το στον υπολογιστή σας.
- Στο GitHub ανοίξτε το repo και πατήστε **Settings → κυλίστε στο τέλος → Danger Zone → Delete this repository**. Μην ανησυχείτε: δεν υπάρχουν δεδομένα αξίας στο παλιό repo (το ιστορικό διασώζεται στο archive/ του νέου πακέτου).
- Δημιουργήστε ξανά repository με το ίδιο όνομα **markos-kapsalos-SOL-TRADE** (Public ή Private όπως θέλετε) — χωρίς κανένα αρχείο αρχικοποίησης.
- Πατήστε **«uploading an existing file»** και σύρετε ΟΛΟ το περιεχόμενο του φακέλου sol_trade/ (όχι το zip — τα αρχεία και τους φακέλους).
- Commit changes και είστε έτοιμοι. Στη συνέχεια ενεργοποιήστε το workflow από το tab Actions (Κεφ. 7).

5.2 Τρόπος Β — Με git, διατηρώντας το ιστορικό commits

Εντολές μετάβασης (Terminal / PowerShell)

```
git clone https://github.com/makiss1956/markos-kapsalos-SOL-TRADE.git
cd markos-kapsalos-SOL-TRADE
git rm -r src config paper_trading scripts docs .github
git rm data/trade_history.csv __init__.py
# Αντίγραψτε εδώ τα πάντα ΜΕΣΑ από τον φάκελο sol_trade/ του νέου πακέτου
git add -A
git commit -m "Αναδιοργάνωση: επαναφορά πακέτου sol_trade, διαγραφή παλιού bot"
git push
```

Αν το git σας ζητήσει ταυτοποίηση, χρησιμοποιήστε Personal Access Token αντί για κωδικό (GitHub → Settings → Developer settings → Tokens). Σε Windows τα εντολές git δουλεύουν ίδια στο PowerShell, αφού έχετε εγκατεστημένο το Git for Windows.

6. Paper trading τοπικά — βήμα προς βήμα

Τοπικά το bot τρέχει συνεχόμενα με βήμα 30 δευτερολέπτων, οπότε φτάνει στα πρώτα σήματα μέσα σε λίγες ώρες αντί για μέρες. Χρειάζεστε Python 3.10 ή νεότερο (στα Windows: python.org → Download → τικάρετε «Add Python to PATH»).

```
Εγκατάσταση και εκκίνηση (Windows / Linux)

# 1) Αποσυμπίεση και είσοδος στον φάκελο
cd sol_trade

# 2) Εικονικό περιβάλλον
python -m venv venv
venv\Scripts\activate # Windows
# source venv/bin/activate # Linux / macOS

# 3) Εξαρτήσεις
pip install -r requirements.txt

# 4) Ρυθμίσεις (προαιρετικό .env για Telegram)
copy .env.example .env # Windows
# cp .env.example .env # Linux / macOS

# 5) ΕΚΚΙΝΗΣΗ paper trading (πραγματικές τιμές, μηδενικό ρίσκο)
python run.py trade
```

Όταν δείτε τη γραμμή «=== sol_trade ξεκίνησε | mode=PAPER ...» το bot δουλεύει. Κάθε 30 δευτερόλεπτα θα εμφανίζεται η τιμή του SOL με τους δείκτες. Μπορείτε να κλείσετε το παράθυρο όποτε θέλετε με Ctrl+C — η κατάσταση αποθηκεύεται αυτόματα και συνεχίζει από εκεί που έμεινε.

- **python run.py status** — υπόλοιπο, ανοιχτή θέση, συναλλαγές μέχρι τώρα.
- **python run.py backtest --days 60** — δοκιμή της στρατηγικής σε 60 ημέρες δεδομένων με πλήρη στατιστικά.
- **python run.py trade --once** — ένα μόνο βήμα (γρήγορος έλεγχος ότι δουλεύει η σύνδεση με το Jupiter).
- **python run.py trade --offline** — εξάσκηση χωρίς internet με συνθετικές τιμές.
- **python run.py reset** — μηδενισμός του paper λογαριασμού στα 1.000 USD.

6.1 Telegram ειδοποιήσεις (προαιρετικό αλλά το συνιστώ)

Στο Telegram: α) μιλήστε στον @BotFather → /newbot → κρατήστε το token. β) Στείλτε οποιοδήποτε μήνυμα στο bot σας. γ) Από τον @userinfobot πάρτε το chat id σας. Βάλτε τα δύο αυτά στο αρχείο .env στα πεδία TELEGRAM_BOT_TOKEN και TELEGRAM_CHAT_ID. Από εκεί και πέρα, κάθε

αγορά/πώληση θα σας έρχεται στο κινητό με την αιτία και το PnL.

7. Paper trading στο GitHub Actions (χωρίς ανοιχτό PC)

Αν θέλετε το bot να δουλεύει ακόμα και με κλειστό υπολογιστή, το repo ήρθε με έτοιμο workflow. Κάθε ώρα τρέχει ένα βήμα με πραγματικές τιμές Jupiter στο δωρεάν lite-api, ενημερώνει το paper_state.json και το price_history.json στο repo, και συνεχίζει σωστά την επόμενη ώρα — κανένας μηδενισμός, καμία απάτη στην ιστορία.

Στοιχείο	Πού	Τι πρέπει να ξέρετε
Ενεργοποίηση	GitHub → tab Actions → «Paper Trading (sol_trade)» → Enable	Μέχρι να το ενεργοποιήσετε, τίποτα δεν τρέχει
Χειροκίνητη εκκίνηση	Ιδιο μέρος → Run workflow	Για να δείτε αμέσως το πρώτο tick
Προγραμματισμός	Κάθε ώρα (cron στο yml)	Τροποποιήσιμο στο αρχείο paper-trading.yml
State	data/paper_state.json + data/price_history.json	Κάνεται commit αυτόματα με [skip ci]
Telegram	Settings → Secrets → TELEGRAM_BOT_TOKEN / TELEGRAM_CHAT_ID	Προαιρετικό — χωρίς secrets απλώς δεν στέλνει
Απενεργοποίηση	Actions → workflow → ⋯ → Disable workflow	Σταματά αμέσως χωρίς να χάνεται το state

Πίνακας 5: Λειτουργία του workflow paper-trading.yml

> **Ρεαλιστικό χρονοδιάγραμμα στο Actions:** η στρατηγική θέλει περίπου 30 σημεία ιστορίας για να ζεσταθούν οι δείκτες. Με ένα tick ανά ώρα, τα πρώτα σήματα εμφανίζονται από τη 2η ημέρα και ένα ουσιαστικό δείγμα (20+ συναλλαγές) χρειάζεται 2-4 εβδομάδες. Για γρηγορότερα αποτελέσματα τρέξτε ταυτόχρονα και τοπικά (Κεφ. 6) — τα δύο scripts δεν εμποδίζουν το ένα το άλλο αν κρατάτε ξεχωριστούς φακέλους.

8. Πώς διαβάζουμε τα αποτελέσματα

Κάθε κλείσιμο θέσης καταγράφεται με αιτία εξόδου: **EMA cross down** (η στρατηγική γύρισε), **RSI υπεραγορά** (το token ανέβηκε απότομα και παίρνουμε κέρδος), **STOP-LOSS** (προσωρινή ζημιά -3% → αυτόματη έξοδος), **TAKE-PROFIT** (+5%) και **TRAILING-STOP** (απώλεια 2% από την κορυφή μετά από κέρδος). Όλα αυτά φαίνονται στο python run.py status και στο data/paper_state.json.

Μετρική	Τι σημαίνει	Τι θεωρώ επιτυχημένο
Win rate	Ποσοστό συναλλαγών με κέρδος	≥ 50% σε 30+ trades
Profit factor	Σύνολο κερδών διαιρεμένο με σύνολο ζημιών	≥ 1,1 (κάτω από 1,0 χάνουμε συνολικά)
Max drawdown	Η μεγαλύτερη πτώση του υπολοίπου από την κορυφή	Επί χαρτού αρκεί < 10%· ρυθμίζεται με stop_loss_pct
Return vs buy & hold	Σύγκριση με το απλό «άγορα και κράτα»	Να κερδίζει ή να ισοφαρίζει με μικρότερο ρίσκο
Αριθμός trades	Στατιστική αξιοπιστία	Τουλάχιστον 30 πριν βγάλουμε συμπέρασμα — τα 5 πρώτα σημαίνουν τίποτα

Πίνακας 6: Μετρικές αξιολόγησης και κριτήρια «θετικών δειγμάτων»

«Δείγματα θετικά» λοιπόν σημαίνει: ύστερα από τουλάχιστον **30 ολοκληρωμένες συναλλαγές** σε πραγματικές τιμές, win rate 50% και άνω, profit factor 1,1 και άνω και max drawdown κάτω από 10%. Τότε και μόνο τότε συζητάμε μετάβαση σε live με μικρό ποσό (Κεφ. 9). Αν αντίθετα η στρατηγική υστερεί, αλλάζουμε παραμέτρους στο config.yaml (π.χ. ταχύτερες EMA 9/21, stop-loss 2%) και ξανακάνουμε backtest πριν ξαναρχίσουμε το paper — ποτέ το αντίστροφο.

9. Η πορεία προς live mode (προσεκτικά)

Το live mode εκτελεί πραγματικά swaps μέσω του Jupiter Swap API v2 με το πορτοφόλι σας. Ενεργοποιείται MONO με την επόμενη εντολή και ζητάει πληκτρολόγηση της λέξης YES ως δικλίδα ασφαλείας:

```
Εντολή live mode (MONO όταν περάσουν τα κριτήρια του Κεφ. 8)

python run.py trade --live
```

Προϋπόθεση	Από πού	Πόσο κοστίζει
JUPITER_API_KEY	developers.jup.ag/portal (δωρεάν εγγραφή)	Δωρεάν σε βασικό επίπεδο
NEO πορτοφόλι μόνο για το bot	Phantom → Create new account	Βάλτε MONO μικρό ποσό (π.χ. 20-50 USDC)

Προϋπόθεση	Από πού	Πόσο κοστίζει
Λίγα SOL για fees	Στο ίδιο πορτοφόλι	0,05+ SOL αρκούν για πολλές συναλλαγές
SOL_PRIVATE_KEY στο .env	Phantom → Export private key	ΔΩΡΕΑΝ — αλλά ποτέ μην το μοιραστείτε ούτε με εμένα
SOLANA_RPC_URL	Προεπιλογή: api.mainnet-beta.solana.com	Δωρεάν, με κάποια rate limits

Πίνακας 7: Κατάλογος προϋποθέσεων live mode

> **Απαράβατοι κανόνες live:** 1) Ποτέ δεν βάλετε περισσότερα από όσα μπορείτε να χάσετε. 2) Ποτέ δεν χρησιμοποιείτε το κύριο πορτοφόλι σας. 3) Το ιδιωτικό κλειδί δεν μπαίνει σε αρχεία που ανεβαίνουν στο GitHub (το .env αγνοείται από το git — μην το μετανομάσετε ποτέ). 4) Πρώτα 2-4 εβδομάδες επιτυχημένου paper σύμφωνα με τον Πίνακα 6.

10. Συχνές ερωτήσεις και προβλήματα

Ε: Το bot τρέχει αλλά δεν ανοίγει θέσεις. Φταίει κάτι;

Α: Όχι — περιμένει φρέσκο EMA crossover, και σε ήρεμη αγορά αυτό μπορεί να αργήσει ώρες. Είναι προστασία, όχι βλάβη. Δείτε το ρυθμό με --offline ή χαμηλώστε τα ema_fast/ema_slow στο config.yaml.

Ε: Δεν μου έρχονται Telegram μηνύματα.

Α: Έλεγξατε και τα δύο πεδία στο .env; Έχετε στείλει πρώτα ένα χειροκίνητο μήνυμα στο bot; Στο GitHub Actions μπήκαν τα δύο Secrets με ΑΚΡΙΒΗ ονόματα TELEGRAM_BOT_TOKEN και TELEGRAM_CHAT_ID;

Ε: Στο Actions βλέπω commit κάθε ώρα. Είναι φυσιολογικό;

Α: Ναι — είναι το state του paper λογαριασμού που αποθηκεύεται. Αν σας ενοχλεί, απενεργοποιήστε το workflow· το state δεν χάνεται.

Ε: Πώς δοκιμάζω άλλο token (π.χ. JUP, BONK);

Α: Στο config.yaml αλλάξτε τα token.mint, token.symbol, token.decimals. Τα mints βρίσκονται αναζητώντας το token στο jup.ag. Κρατήστε quote: USDC.

Ε: Έχασα τα πάντα με update; Μπορώ να μηδενίσω το paper;

Α: python run.py reset — γυρνάει στα 1.000 USD και σβήνει το ιστορικό συναλλαγών (όχι το ιστορικό τιμών· αν θέλετε πλήρες καθάρισμα, σβήστε και το data/price_history.json).

Ε: Το Actions απέτυχε με ModuleNotFoundError.

Α: Σημαίνει ότι το upload δεν ολοκληρώθηκε — ελέγξτε ότι ο φάκελος sol_trade/ με τα 16 αρχεία υπάρχει στο repo και ότι το run.py είναι στη ρίζα.

Ε: Μπορώ να τρέχω τοπικά ΚΑΙ Actions ταυτόχρονα;

Α: Ναι, με ξεχωριστούς φακέλους. Μοιράζονται το ίδιο Jupiter API χωρίς πρόβλημα — το δωρεάν tier αντέχει άνετα και τα δύο.

Ε: Πόσο κοστίζει το paper trading;

Α: Τίποτα. Χρησιμοποιεί μόνο το δωρεάν Jupiter Price API (lite-api.jup.ag) και στα Actions δωρεάν minutes (ένα public repo έχει απεριόριστα).

> *Με τη νέα δομή το έργο σας επιτέλους δουλεύει όπως σχεδιάστηκε: ένας καθαρός, δοκιμασμένος κώδικας, ένας μοναδικός τρόπος εκτέλεσης και πλήρης καταγραφή. Καλή επιτυχία με τα πρώτα θετικά δείγματα — και θυμηθείτε: το επί χαρτού στάδιο υπάρχει ακριβώς για να κάνουμε τα λάθη δωρεάν.*